

 Politechnika Wrocławska	Projektowanie efektywnych algorytmów – projekt - zadanie 2	
	Maksymilian Tara 264000	Data oddania sprawozdania: 4.11.2023.
	Poniedziałek 15:15 – 16:55	Dr inż. Jarosław Mierzwa

Spis Treści

1.	Wstęp	2
2.	Omówienie Algorytmu	2
2.1.	Opis Działania Algorytmu	3
3.	Opis Implementacji Algorytmu i Klas w Programie	4
3.1.	Klasa main	5
3.2.	Klasa Matrix	5
3.3.	Klasa ReadPathFromFile	5
3.4.	Klasa SimAnnealing	5
3.5.	Opis Implementacji	5
4.	Plan Eksperymentu	7
5.	Wyniki Testów i Omówienie	7
5.1.	Graf mały – ftv55.atasp	7
5.2.	Graf średni – ftv170.atasp	10
5.3.	Graf duży – rbg358.atasp	13
6.	Najlepsze Wyniki dla Każdego Grafu	15

1. Wstęp

Rozpatrywanym problemem, jest problem komiwojażera (ang. TSP – Travelling Salesman Problem). Polega on, na znalezieniu minimalnego cyklu Hamiltona w pełnym grafie ważony. Cykl Hamiltona to taki cykl, w którym w grafie każdy wierzchołek odwiedzany jest dokładnie raz, a następnie następuje powrót do wierzchołka startowego. Minimalny – oznacza, że wagi przypisane do krawędzi grafu (połączeń między wierzchołkami) przez które przechodzi nasz znaleziony cykl, mają dawać najmniejszą możliwą sumę. Problem ten można przedstawić na przykładzie z życia jako podróżującego sprzedawcę, który musi odwiedzić każde z n miast na swojej określonej drodze (zaczyna z punktu A i na końcu chce znaleźć się w punkcie B) jak najniższym kosztem.

W tym eksperymencie problem rozpatrywany będzie dla grafów asymetrycznych – waga ścieżki (długość lub koszt drogi) z wierzchołka A do wierzchołka B, może być inna niż z wierzchołka B do wierzchołka A (w drugą stronę).

Problem TSP jest problemem NP-trudnym.

W celu rozwiązania tego problemu został zaimplementowany algorytm Symulowanego Wyżarzania (ang. Simulated Annealing algorithm), z 3 różnymi współczynnikami schładzania α dla każdego sprawdzanego grafu.

2. Omówienie Algorytmu

Algorytm Symulowanego Wyżarzania to algorytm heurystyczny – to znaczy taki, który daje rozwiązania zbliżone do optymalnego, a nie optymalne. Ma on jednak znacznie mniejsze zapotrzebowanie na moc obliczeniową, pamięć i wykonuje się w znacznie krótszym czasie. Dla grafów o dużej ilości wierzchołków znalezienie rozwiązania za pomocą tradycyjnego algorytmu może wymagać zbyt dużej ilości zasobów lub czasu, bądź może być zwyczajnie niemożliwe w realnym czasie. Przykładowo, jeden z rozważanych grafów w eksperymencie z algorytmem Symulowanego Wyżarzania posiada 358 wierzchołków, co daje nam $(358-1)!$ możliwych permutacji ścieżek. Na podstawie zmierzonego czasu działania algorytmu Przeglądu Zupełnego (ang. Brute Force) rozważanego w poprzednim zadaniu, możemy oszacować czas potrzebny na wykonanie go na tym samym urządzeniu dla grafu o 358 wierzchołkach – zakładając oczywiście, że urządzenie będzie miało wystarczająco dużo pamięci do wykorzystania.

Weźmy zatem czas wykonania algorytmu Brute Force dla grafu o 13 wierzchołkach - 16655319 μ s. Graf ten posiadał $(13-1)!$ permutacji ścieżek. Możemy zatem na podstawie liczby permutacji oszacować czas potrzebny do sprawdzenia wszystkich permutacji dla grafu o wielkości 358:

$$x = \frac{(358 - 1)! * 16655319}{(13 - 1)!} \approx 2.9933983974213323782365632765765e + 756 \mu s$$

Po przeliczeniu na łatwiejszą do zwizualizowania dla nas jednostkę czasu otrzymujemy 9.4920040506764725337283208922389e+742 lat (!!!). Dla porównania według aktualnych ustaleń naukowców Wszechświat powstał w Wielkim Wybuchu około 13.8 miliarda lat temu, co można również zapisać jako $1.38e + 10$ lat.

Można zatem zauważyć, że znalezienie optymalnego rozwiązania dla tego grafu przy użyciu algorytmu Przeglądu Zupełnego na normalnym komputerze nie jest możliwe. Jednak za pomocą algorytmu metaheurystycznego, rozwiązanie zbliżone do optymalnego możemy otrzymać już po kilku minutach. Znając więc nakład obliczeniowy dla „zwykłych” algorytmów, drobna utrata na optymalności wyniku wydaje się łatwym do zaakceptowania minusem.

Algorytmy metaheurystyczne mają pewne cechy i założenia:

- Celem ich działania jest efektywne przeszukiwanie przestrzeni
 - Mają znajdować dobre rozwiązania w określonym regionie
 - Oraz przeszukiwać możliwie największy obszar przestrzeni problemu
- Dają wyniki przybliżone
- Są niedeterministyczne – dla tych samych danych początkowych możemy przy każdym uruchomieniu uzyskać inny wynik
- Posiadają mechanizmy zabezpieczające przed utknięciem w ograniczonym obszarze przestrzeni problemu
- Do wyszukiwania kolejnych rozwiązań wykorzystują dane zgromadzone w poprzednich etapach działania

Algorytmy te często są inspirowane zjawiskami występującymi w naturze, nauce lub społeczeństwie. Sam algorytm Symulowanego Wyżarzania był inspirowany termodynamiką, a konkretnie rozkładem Boltzmanna: $N(E) = \exp\left(-\frac{\Delta E}{kT}\right)$.

2.1. Opis Działania Algorytmu

Sam algorytm operuje następująco:

1. Znajdywane jest rozwiązanie początkowe przy użyciu metody zachłannej.
2. Zaczynamy wykonywanie algorytmu Symulowanego Wyżarzania dla ustalonych parametrów i poprzednio znalezionej najlepszej ścieżki.
3. Przeszukujemy sąsiedztwo (wykonujemy ruch).
4. Obliczamy całkowity koszt dla nowo znalezionej ścieżki.
5. Porównujemy go z poprzednim.
6. Jeśli jest lepszy (mniejszy) zapisujemy ścieżkę.
7. Wykorzystujemy mechanizm zabezpieczający przed utknięciem w jednym obszarze przestrzeni problemu (minimum lokalnym). W zależności od jego rezultatu, zapisujemy ścieżkę nawet jeżeli jej koszt jest gorszy od poprzedniej (większy).
8. Schładzamy układ – zmniejszamy temperaturę.
9. Wracamy do kroku nr 3, jeśli nie upłynął założony czas działania algorytmu.

Rozwijając poszczególne pojęcia:

- Metoda zachłanna - wybieramy wierzchołek startowy, wybieramy wychodzącą z niego ścieżkę o najmniejszej wadze. To samo robimy dla wierzchołka, do którego prowadziła wybrana ścieżka, aż do zakończenia algorytmu, czyli odwiedzenia każdego wierzchołka 1 raz – musimy gdzieś przechowywać odwiedzone już wierzchołki, w napisanym przeze mnie

programie odpowiada za to tablica zmiennych typu bool. 0 oznacza, że wierzchołek nie został jeszcze odwiedzony, a 1, że był już odwiedzony i nie możemy ponownie wybrać ścieżki do niego prowadzącej. Po odwiedzeniu wszystkich wierzchołków wracamy ponownie do wierzchołka startowego. Otrzymane rozwiązanie zapisujemy (ścieżkę oraz jej całkowity koszt).

- Parametry startowe algorytmu to:
 - Temperatura startowa – startowy poziom „energii”, zależny od wielkości grafu.
$$\text{Temperatura} = 100 * \text{ilość wierzchołków}.$$

Wzór ten został wybrany za pomocą eksperymentów. Na początku temperatura startowa podawana była jako stałe liczby w kodzie. Po przetestowaniu działania algorytmu wiele razy zauważyłem, że temperatury startowe dla których algorytm daje akceptowalne wyniki, są w tym samym rzędzie wielkości co liczba otrzymana po przemnożeniu liczby wierzchołków w grafie przez 100 i są do niej zbliżone.
 - Współczynnik schładzania α – stała liczba, od której zależy jak szybko temperatura będzie się zmniejszać wraz z działaniem algorytmu.
Nie jest możliwe znalezienie takiego α , które jednocześnie będzie optymalne dla wszystkich grafów. Więc dla każdego grafu optymalne α zostało dobrane osobno, również metodą eksperymentalną. Jednak dla każdego grafu zostało sprawdzone działanie algorytmu dla 3 różnych współczynników α .
 - Wzór na schładzanie – wzór, który określa sposób schładzania, wykorzystujący wyżej opisany współczynnik α do zmniejszenia temperatury w odpowiednim momencie działania algorytmu.
$$\text{Temperatura} = \text{Temperatura} * \alpha.$$
 - Czas działania algorytmu – wartość czasu w sekundach podawana przed uruchomieniem algorytmu. Określa po jakim czasie algorytm ma się zatrzymać. Dla małych grafów mogą być to np. 2 minuty (120 sekund), dla średnich 4 minuty (240 sekund) a dla dużych 6 minut (360 sekund).
- Przeszukiwanie sąsiedztwa (ruch) – sposób dobierania ścieżki sąsiedniej do tej, w której akurat się znajdujemy. W tym przypadku nowa ścieżka dobierana jest następująco:
 - Losowane są 2 wierzchołki z aktualnej ścieżki (poza startowym i końcowym).
 - 2 wylosowane uprzednio wierzchołki zamieniane są miejscami.
- Mechanizm zabezpieczający przed utknięciem w minimum lokalnym – mechanizm umożliwiający przyjęcie nowej ścieżki jako lepszej (następne kroki algorytmu będą wykonywane względem tej ścieżki) nawet jeżeli ma ona wyższy koszt całkowity.
Ta część algorytmu inspirowana jest wzorem na rozkład Boltzmanna:
 - Prawdopodobieństwo obliczane jest ze wzoru $e^{-(\Delta T)/T}$, gdzie T = temperatura, a ΔT = koszt ścieżki sąsiedniej – koszt aktualnej ścieżki
 - Jeśli jest ono większe niż losowo dobrana liczba z zakresu $<0, 1>$ ścieżka zostanie przyjęta.
- Schładzanie układu odbywa się według wzoru na schładzanie.

3. Opis Implementacji Algorytmu i Klas w Programie

3.1. Klasa main

W tej klasie znajduje się menu, z którego sterujemy całym programem i wywoływane są poszczególne funkcje programu, takie jak wczytanie grafu, podanie parametrów startowych i uruchomienie algorytmu.

3.2. Klasa Matrix

W tej klasie znajdują się funkcje umożliwiające wczytanie grafu z pliku .atsp do macierzy sąsiedztwa oraz ilości jego wierzchołków.

3.3. Klasa ReadPathFromFile

Klasa ta zawiera metody pozwalające na wczytanie z pliku .txt ścieżki w grafie i znalezienie jej całkowitego kosztu na aktualnie wczytanym grafie.

3.4. Klasa SimAnnealing

Ta Klasa zawiera wszystkie funkcje potrzebne do działania algorytmu, jak i również wyświetlanie danych po jego zakończeniu, zapis wyników do pliku i funkcje do przeprowadzania zautomatyzowanych testów.

3.5. Opis Implementacji

Program został napisany w języku C++ i stosował się do podanych zaleceń:

- Wykorzystywane struktury danych są alokowane dynamiczne, w zależności do rozmiaru problemu,
- Rozwiązanie początkowe generowane jest przy użyciu metody zachłannej,
- Implementacja algorytmu została wykonana zgodnie z obiektowym paradygmatem programowania,
- W trakcie działania algorytmu pamiętane jest najlepsze dotychczas znalezione rozwiązanie
- Menu programu umożliwia:
 - Wczytanie danych z pliku,
 - Wybór kryterium zatrzymania algorytmu jako czasu w sekundach,
 - Wybór współczynnika chłodzenia dla algorytmu Symulowanego Wyżarzania,
 - Uruchomienie algorytmu Symulowanego Wyżarzania,
 - Zapis ścieżki znalezionego rozwiązania do pliku .txt (dzieje się automatycznie po zakończeniu działania algorytmu),
 - Wczytanie ścieżki rozwiązania z pliku .txt i obliczenie na podstawie wczytanego grafu i ścieżki, jej kosztu.
- Program po wykonaniu algorytmu wyświetla następujące dane (wyniki):
 - Temperaturę końcową,
 - Wartość wyrażenia $\exp(-1/T_k)$, gdzie T_k = Temperatura końcowa, czyli prawdopodobieństwo przyjęcia gorszego rozwiązania w momencie zakończenia algorytmu,

- Współczynnik chłodzenia
- Najlepsza znaleziona ścieżka
- Koszt najlepszej ścieżki
- Czas działania algorytmu, po którym znaleziono najlepsze dotychczasowe rozwiązanie w sekundach.
- Implementacja algorytmu wykorzystuje tylko podstawowe struktury danych (dynamicznie alokowane tablice),
- Raz ustawione parametry są zachowywane, żeby nie trzeba było podawać ich ponownie przy każdym wykonaniu algorytmu,
- Sposób chłodzenia realizowany jest według wzoru $T = a * T$,
- Sąsiedztwo znajduwane jest przez zamianę miejscami losowych 2 wierzchołków w ścieżce,
- Temperatura początkowa obliczana jest dla każdego grafu z osobna,
- Zaimplementowano 3 schematy schładzania (różne współczynniki schładzania a),
- Dla każdego grafu został dobrany 1 współczynnik a , który wydaje się najbardziej optymalny, oraz 2 inne, dla których również zostały wykonane testy.

Implementacja algorytmu wymagała następujących struktur danych:

- Graf był reprezentowany jako macierz sąsiedztwa – do jej przechowania potrzebna była dwuwymiarowa macierz wag o rozmiarze (liczba wierzchołków * liczba wierzchołków)
- Ścieżki znajduwane przez algorytm reprezentowane były za pomocą jednowymiarowych tablic – 3 tablice o rozmiarze (liczba wierzchołków + 1)

Do zaimplementowania elementów losowych wykorzystana została biblioteka `<random>`.

Przykładowe użycie w funkcji odpowiadającej za przeszukiwanie sąsiedztwa:

```
void SimAnnealing::swapRandomCities() {
    //Losowanie z miast
    std::random_device rng;
    std::mt19937 gen( sd: rng());
    std::uniform_int_distribution<> distribution( a: 1, b: numberOfCities-1);
    int swap1 = distribution( &: gen);
    int swap2 = distribution( &: gen);
    //Zabezpieczenie zeby nie wylosowały sie 2 te same
    while(swap2 == swap1){
        swap2 = distribution( &: gen);
    }
}
```

Do zaimplementowania czasowego warunku zatrzymania algorytmu i zapisywania momentu, w którym zostało znalezione najlepsze rozwiązanie wykorzystana została biblioteka `<chrono>`.

Przykładowe użycie do zapisu momentu znalezienia najlepszej ścieżki:

```

if (currentDistance < bestDistance){
    for(int j = 0; j < numberOfCities + 1; j++){
        bestSolution[j] = currentSolution[j];
    }
    bestDistance = currentDistance;
    BestSolutionFoundTime = std::chrono::duration_cast<std::chrono::seconds>( d: std::chrono::system_clock::now() - startTime).count();
}

```

4. Plan Eksperymentu

Grafy, dla których próbowano znaleźć jak najlepsze rozwiązanie zapisane są w plikach:

- ftv55.atsp
- ftv170.atsp
- rgb358.atsp

Proponowane czasy działania algorytmów to kolejno:

- 2 minuty (120 sekund) dla grafu małego – ftv55
- 4 minuty (240 sekund) dla grafu średniego – ftv170
- 6 minut (360 sekund) dla grafu dużego – rgb358

Parametry początkowe: Dla różnych grafów zmieniane było kryterium stopu (czas działania), temperatura startowa (zmienia się sama, ponieważ jest liczona wzorem w zależności od grafu), oraz współczynnik chłodzenia α .

Parametrem wspólnym dla wszystkich grafów była liczba iteracji wykonywana przed każdym chłodzeniem. Wynosiła ona 1000.

Każdy z grafów został przetestowany dla odpowiadającego mu czasu działania i temperatury startowej, oraz dla 3 współczynników schładzania α :

- 0.999
- 0.99935
- 0.9995

Dla każdego grafu i każdego współczynnika schładzania α zostało wykonane 10 testów. Na podstawie wyników testów, obliczone zostały błędy względne w stosunku do najbardziej optymalnej ścieżki znalezionej dla tego grafu, a same wyniki przedstawione zostały w tabelach.

W sprawozdaniu umieszczono również najlepszą znaną ścieżkę dla każdego z grafów.

Optymalność współczynnika schładzania dla grafu oceniana była na podstawie temperatury końcowej oraz prawdopodobieństwa przyjęcia gorszego rozwiązania. Powinno wynosić ono maksymalnie kilka %, a minimalnie kilka setnych % dla optymalnego współczynnika α .

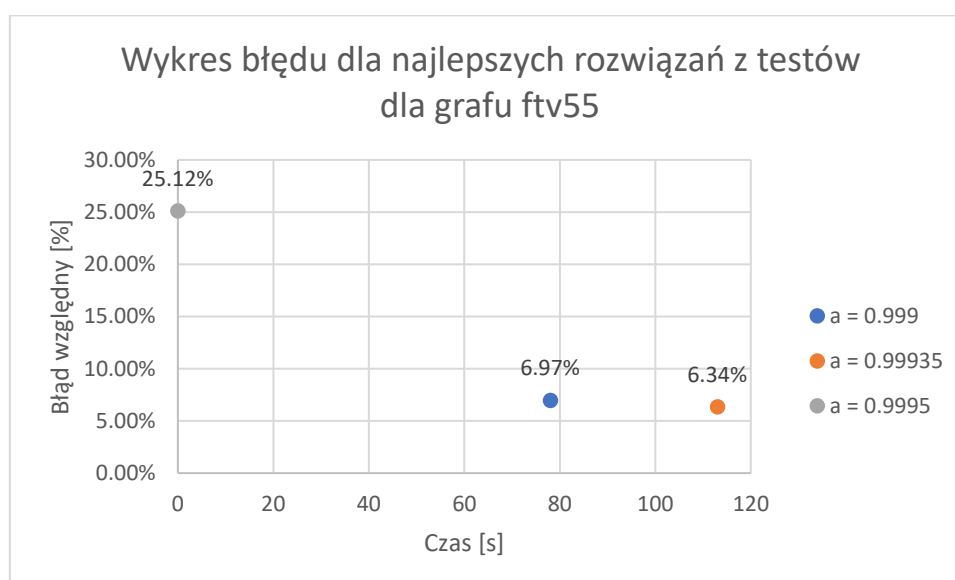
Wierzchołkiem startowym dla każdego grafu jest wierzchołek 0.

5. Wyniki Testów i Omówienie

5.1. Graf mały – ftv55.atsp

Na podstawie powyższych wyników i znanego najlepszego rozwiązania dla tego grafu (1608) możemy obliczyć błąd względny dla każdego pomiaru.

Rozmiar grafu	56									
Czas Wykonywania	120 s									
a	0.999									
Błąd	14.93 %	10.26 %	10.20 %	8.96 %	12.69 %	6.97 %	10.88 %	13.12 %	12.94 %	8.21 %
a	0.99935									
Błąd	9.02 %	16.92 %	14.74 %	6.34 %	15.86 %	10.76 %	7.77 %	10.32 %	12.06 %	7.52 %
a	0.9995									
Błąd	25.12 %	25.12 %	25.12 %	25.12 %	25.12 %	25.12 %	25.12 %	25.12 %	25.12 %	25.12 %



Rozwiązaniem otrzymywanym przy użyciu metody zachłannej w tym grafie było 2012.

Na podstawie samego wykresu można by wywnioskować, że optymalnym współczynnikiem schładzania dla tego grafu jest $a = 0.99935$. Jest to jednak niezgodne z prawdą. Jeżeli popatrzymy na wyniki testów zamieszczone w tabelach zauważymy, że Temperatura końcowa oraz prawdopodobieństwo akceptacji gorszego wyniku po zakończeniu algorytmu jest bardzo wysokie dla współczynnika $a = 0.99935$, wynoszą one przykładowo $T_k = 8.77$ i $\exp(-1/T_k) = 0.8923$. Oznacza to, że dla tego współczynnika algorytm kończył działanie, kiedy prawdopodobieństwo przyjęcia gorszego wyniku (mechanizm uciekania z minima lokalnego) wynosiło 89%! Algorytm nie miał zatem czasu się schłodzić i przyjmował pierwszą lepszą ścieżkę, na którą się natknął. Przy tak wysokiej temperaturze częstym zjawiskiem jest skorzystanie z mechanizmu ucieczki z minimów lokalnych, a dopiero przy mniejszych temperaturach zaczyna się szukanie właściwego rozwiązania.

Natomiast jeżeli przyjrzymy się danym dla współczynnika $a = 0.999$ widzimy, że algorytm schładzał się praktycznie idealnie wraz z czasem jego działania. Temperatura i prawdopodobieństwo końcowe nie są za duże ani za małe – algorytm miał czas uciec z minimów lokalnych jak i znaleźć najlepszy

dostępny wynik w czasie jego działania. Jeśli następowałoby dalsze chłodzenie (algorytm działałby dłużej dla tego współczynnika) i co za tym idzie prawdopodobieństwo zmniejszałoby się jeszcze bardziej, algorytm wykonywałby niepotrzebne ruchy, ponieważ najlepsze rozwiązania w miejscu, w którym akurat się znajduje zostałyby już dawno znalezione.

W rozważaniach, dlaczego zatem wynik dla współczynnika $a = 0.99935$ wydaje się lepszy – błąd względny dla najlepszego rozwiązania na wykresie jest najmniejszy, możemy również wziąć pod uwagę średni błąd ze wszystkich testów:

0.999	10.91%
0.99935	11.13%
0.9995	25.12%

Na podstawie wszystkich zebranych danych, widzimy zatem, że znalezienie najlepszego rozwiązania ze wszystkich, dla współczynnika $a = 0.99935$, wynika z losowości algorytmu w dobieraniu sąsiedztwa i jest kwestią przypadku. Optymalnym współczynnikiem a dla tego algorytmu był zatem 0.999.

Współczynnik 0.9995 zaś jest zdecydowanie za duży, ponieważ algorytm przez cały czas jego działania, nie schładza się praktycznie w ogóle. Końcowe prawdopodobieństwo wynosi ponad 97% dla każdego z testów co oznacza, że algorytm nie miał nawet szansy znaleźć lepszego wyniku niż metoda zachłanna. Zatem najlepszym rozwiązaniem dla tego współczynnika jest właśnie rozwiązanie z metody zachłannej, które na wykresie jest przedstawione jako znalezione po 0 sekundach, ponieważ z takiej ścieżki algorytm startował. Jest to zdecydowanie najgorszy z dobranych do testów współczynników dla tego grafu.

5.2. Graf średni – ftv170.atsp

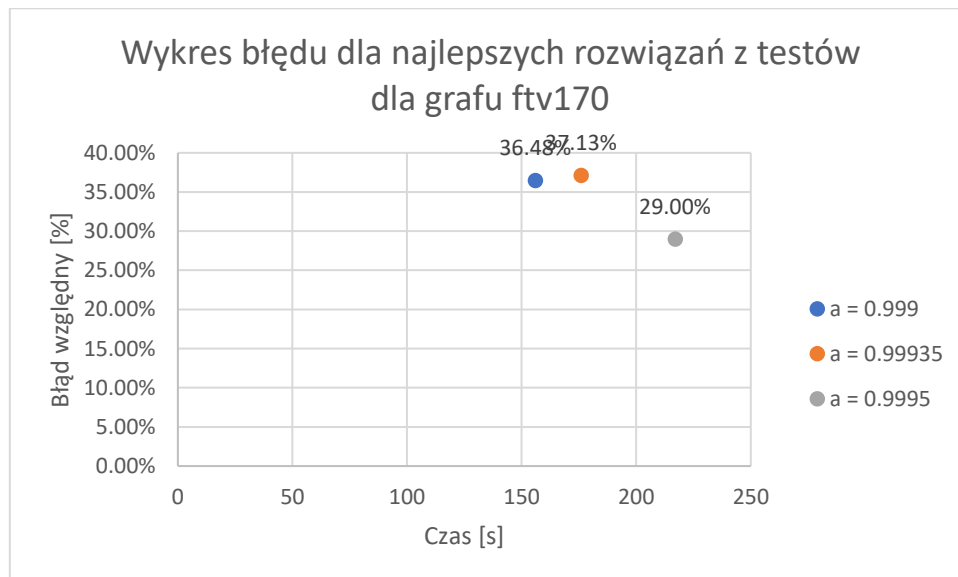
Rozmiar grafu	171									
a	0.999									
Tk	0.000 189	0.000 232	0.000 221	0.000 218	0.000 223	0.000 214	0.000 234	0.000 229	0.000 234	0.000 23
exp(-1/Tk)	0	0	0	0	0	0	0	0	0	0
koszt	3923	3923	3923	3760	3923	3923	3923	3923	3923	3923
czas znalezienia	0	0	0	156	0	0	0	0	0	0

Rozmiar grafu	171									
a	0.99935									
Tk	0.122 216	0.132 909	0.130 766	0.130 342	0.124 866	0.127 575	0.125 681	0.129 581	0.131 192	0.126 008
exp(-1/Tk)	0.000 28	0.000 54	0.000 477	0.000 466	0.000 333	0.000 394	0.000 35	0.000 445	0.000 489	0.000 358
koszt	3889	3923	3811	3923	3778	3869	3902	3923	3783	3923
czas znalezienia	176	0	176	0	176	174	177	0	165	0

Rozmiar grafu	171									
a	0.9995									
Tk	2.205 81	2.153 49	2.287 83	2.203 6	2.202 5	2.188 23	2.216 87	2.257 14	2.099 25	2.013 9
exp(-1/Tk)	0.635 497	0.628 535	0.645 91	0.635 209	0.635 064	0.633 186	0.636 936	0.642 083	0.621 04	0.608 627
koszt	3603	3419	3886	3819	3632	3919	3841	3923	3841	3554
czas znalezienia	229	215	224	224	234	228	232	0	225	217

Na podstawie powyższych wyników i znanego najlepszego rozwiązania dla tego grafu (2755) możemy obliczyć błąd względny dla każdego pomiaru.

Rozmiar grafu	171									
Czas Wykonywania	240 s									
a	0.999									
Błąd	42.40 %	42.40 %	42.40 %	36.48 %	42.40 %	42.40 %	42.40 %	42.40 %	42.40 %	42.40 %
a	0.99935									
Błąd	41.16 %	42.40 %	38.33 %	42.40 %	37.13 %	40.44 %	41.63 %	42.40 %	37.31 %	42.40 %
a	0.9995									
Błąd	30.78 %	24.10 %	41.05 %	38.62 %	31.83 %	42.25 %	39.42 %	42.40 %	39.42 %	29.00 %



Rozwiązaniem otrzymywanym przy użyciu metody zachłannej w tym grafie było 3923.

Przed zaczęciem testów zakładałem, że optymalnym współczynnikiem a dla tego grafu będzie 0.99935. Jednak na wykresie widzimy, że dało ono najgorszy z najlepszych wyników. Nie dziwne jest,

że wynik ten może być gorszy niż dla $a = 0.999$, ponieważ wtedy algorytm chłodzi się tak szybko, że większość czasu jego działania spędzona jest na przyjmowaniu ścieżek wyłącznie lepszych – końcowe prawdopodobieństwo jest tak małe, że nie da się go zapisać za pomocą zmiennej typu double i jest ono zapisywane jako 0. Jednak nie można uznać $a = 0.999$ za optymalne ze względu właśnie na zbyt szybko przebiegające schładzanie. Dla tego grafu algorytm startujący z tym współczynnikiem będzie w stanie znaleźć tylko minimum lokalne i nie ma opcji wyjścia z niego co widzimy na danych w tabeli – większość najlepszych rozwiązań jest równa rozwiązaniu z metody zachłannej. Pojedyncze lepsze rozwiązanie jest więc kwestią losowości metody dobierania sąsiedztwa.

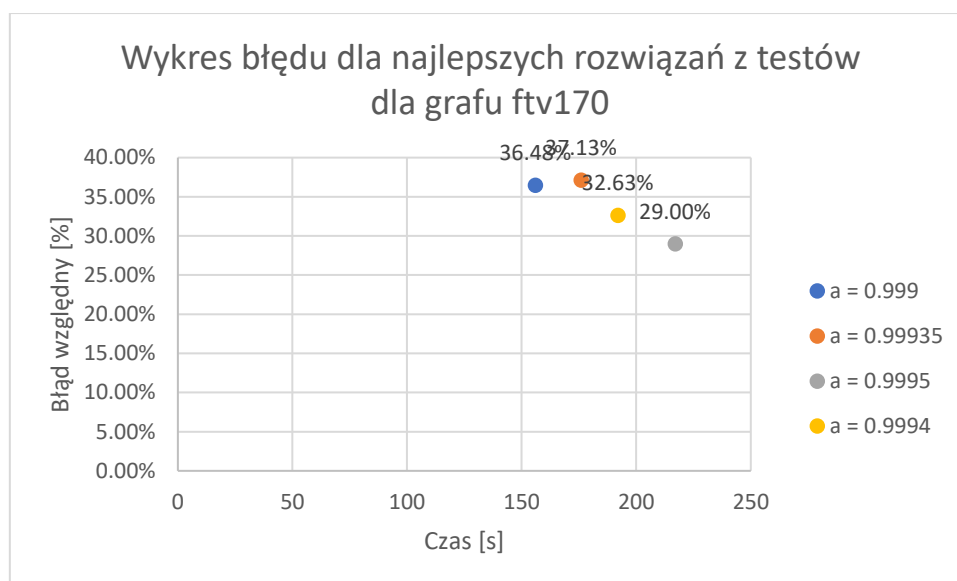
Potwierdza to dużo lepszy wynik dla współczynnika $a = 0.9995$, który jednak również moim zdaniem nie może zostać uznany za optymalny, ponieważ schładzanie przy nim przebiega zbyt wolno – końcowe prawdopodobieństwo wynosi w tym przypadku około 63%. Oznacza to, że znalezienie najlepszego wyniku przy użyciu tego współczynnika jest znowu kwestią przypadku wynikającą z losowości metody dobierania sąsiedztwa.

Zostaje nam zatem współczynnik $a = 0.99935$. Ma on najlepsze ze wszystkich 3 współczynników temperatury i prawdopodobieństwa końcowe, jednak wyniki przez niego znajduwane nie są satysfakcjonujące. Uznałem zatem, że optymalne a będzie znajdowało się gdzieś pomiędzy 0.99935 a 0.9995, i przeprowadziłem dalsze testy w celu jego znalezienia:

Rozmiar grafu	171									
a	0.9994									
Tk	0.334 199	0.323 93	0.289 714	0.328 432	0.341 703	0.353 807	0.358 725	0.346 453	0.277 294	0.318 914
$\exp(-1/Tk)$	0.050 175	0.045 635	0.031 692	0.047 607	0.053 583	0.059 226	0.061 566	0.055 777	0.027 153	0.043 472
koszt	3775	3749	3702	3677	3654	3707	3734	3837	3683	3735
czas znalezienia	186	188	183	182	192	180	186	195	187	197

Rozmiar grafu	171									
Czas Wykonywania	240 s									
a	0.9994									
Błąd	37.02 %	36.08 %	34.37 %	33.47 %	32.63 %	34.56 %	35.54 %	39.27 %	33.68 %	35.57 %

Dla współczynnika 0.9994 otrzymujemy do tej pory najlepsze temperatury końcowe i co za tym idzie prawdopodobieństwa. Możemy również zestawić błąd najlepszego znalezionej przez niego rozwiązania na wykresie wraz z innymi współczynnikami:



Możemy zaobserwować, że daje on lepsze wyniki niż $a = 0.99935$, które wydawało się wcześniej optymalne. Jednak dalej są one gorsze niż dla współczynnika 0.9995. Możliwe jest, że graf utyka w minimum lokalnym pomimo tego, że parametry wskazują na optymalnie dobrany współczynnik schładzania. Rozwiązaniem tego problemu mogłoby być uznanie grafu za graf duży i co za tym idzie zwiększenie czasu wykonywania się go do 6 minut oraz skorzystanie ze współczynnika 0.9995.

5.3. Graf duży – rbg358.atsp

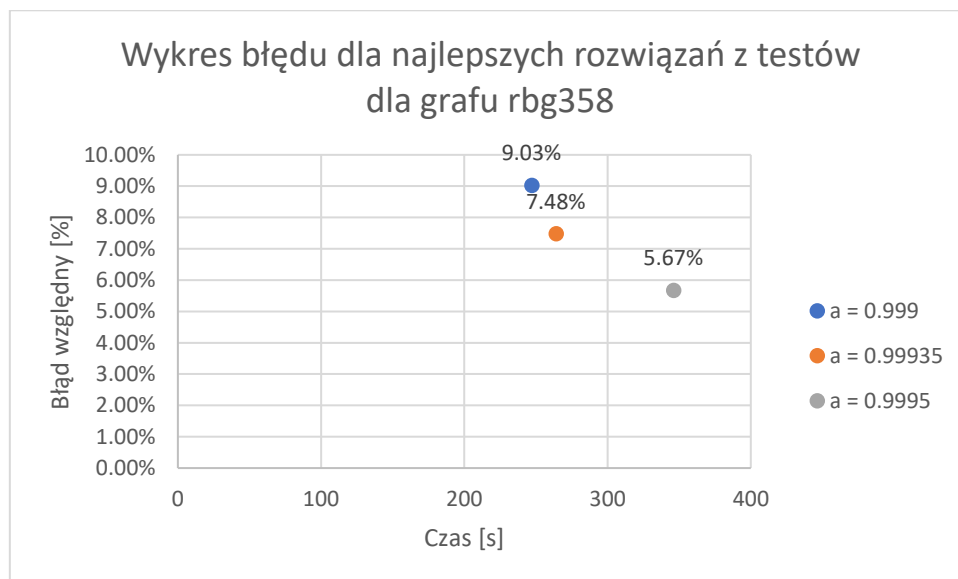
Rozmiar grafu	358									
a	0.999									
Tk	7.15E-07	6.16E-07	1.03E-06	1.39E-06	8.18E-07	8.27E-07	7.91E-07	8.44E-07	7.87E-07	7.96E-07
exp(-1/Tk)	0	0	0	0	0	0	0	0	0	0
koszt	1323	1292	1318	1275	1308	1280	1287	1304	1305	1268
czas znalezienia	181	337	179	192	214	292	182	184	176	247

Rozmiar grafu	358									
a	0.99935									
Tk	0.005056	0.005261	0.005176	0.004757	0.005086	0.005538	0.005156	0.005474	0.005146	0.005431
exp(-1/Tk)	1.28E-86	2.81E-83	1.24E-84	4.95E-92	4.07E-86	3.81E-79	5.85E-85	4.55E-80	4.01E-85	1.09E-80
koszt	1273	1298	1280	1292	1250	1274	1274	1282	1297	1277
czas znalezienia	280	267	329	264	264	280	281	313	292	311

Rozmiar grafu	358									
a	0.9995									
Tk	0.218 101	0.235 444	0.216 146	0.228 37	0.227 458	0.221 287	0.215 822	0.216 904	0.215 175	0.217 121
exp(-1/Tk)	0.010 203	0.014 303	0.009 789	0.012 54	0.012 322	0.010 9	0.009 721	0.009 948	0.009 587	0.009 994
koszt	1256	1246	1284	1272	1321	1229	1250	1288	1303	1263
czas znalezenia	349	355	351	350	353	346	350	354	344	349

Na podstawie powyższych wyników i znanego najlepszego rozwiązania dla tego grafu (1163) możemy obliczyć błąd względny dla każdego pomiaru.

Rozmiar grafu	358									
Czas Wykonywania	360 s									
a	0.999									
Błąd	13.76 %	11.09 %	13.33 %	9.63%	12.47 %	10.06 %	10.66 %	12.12 %	12.21 %	9.03 %
a	0.99935									
Błąd	9.46%	11.61 %	10.06 %	11.09 %	7.48%	9.54%	9.54%	10.23 %	11.52 %	9.80 %
a	0.9995									
Błąd	8.00%	7.14%	10.40 %	9.37%	13.59 %	5.67%	7.48%	10.75 %	12.04 %	8.60 %



Rozwiązaniem otrzymywanym przy użyciu metody zachłannej w tym grafie było 1812.

W przypadku tego grafu wnioski, które mogą zostać wyciągnięte z tabel zarówno jak i z wykresu są takie same. Optymalnym współczynnikiem dla tego grafu jest $a = 0.9995$ – daje on najlepsze wyniki i jednocześnie zapewnia najlepsze chłodzenie w trakcie działania algorytmu. Nie utyka on w minimum lokalnym, ale też nie chłodzi się za szybko, temperatura i prawdopodobieństwo końcowe (około 1%) są bardzo dobre.

Dla pozostałych współczynników chłodzenie postępuje za szybko, algorytm zostaje w minimum lokalnym i większość czasu wykonuje ruchy nie wnoszące niczego, ponieważ najlepsze rozwiązanie dla tego minimum zostało już znalezione, a przy tak niskiej temperaturze nie zaakceptuje on gorszej ścieżki w celu wyjścia z tego minimum.

6. Najlepsze Wyniki dla Każdego Grafu

W celu znalezienia jak najlepszego wyniku dla każdego grafu, została napisana nowa wersja metody zachłannej znajdowania ścieżki startowej. Znajduje ona najlepszą ścieżkę dla każdego możliwego wierzchołka startowego w grafie, a następnie wybiera najlepszą z tych wszystkich ścieżek jako startową dla algorytmu.

Nie udało się jednak przy pomocy tej metody znaleźć lepszych rozwiązań dla żadnego z grafów niż te znalezione w testach. Dla 358 znaleziono lepsze rozwiązanie jednak również zaczynające od wierzchołka 0.

Ścieżka dla grafu ftv55.atsp o długości 1710:

56

0 33 14 15 17 16 12 39 11 38 37 36 9 32 8 7 10 51 13 35 4 6 5 47 48 3 31 55 34 1 2 25 24 18 19 40 20
42 21 22 41 50 23 54 27 49 45 44 28 53 43 30 46 29 26 52 0

Ścieżka dla grafu ftv170.atsp o długości 3419:

171

0 2 1 112 113 127 126 125 129 128 131 132 111 110 114 109 107 106 105 97 98 95 96 165 163 99
100 101 123 162 122 121 120 102 103 104 164 135 138 139 140 141 134 115 116 117 118 119 124
146 145 6 7 8 14 13 17 21 29 22 15 159 16 19 20 32 31 30 28 27 26 23 24 25 150 161 160 151 152
142 149 148 144 143 147 137 136 130 9 10 76 74 75 11 12 18 158 36 157 33 41 155 42 45 46 47 48
51 52 53 43 55 54 58 59 49 170 168 72 71 60 68 85 84 69 67 167 70 66 63 64 56 57 62 61 65 88 153
154 89 90 94 166 108 93 92 91 87 86 83 50 38 39 44 40 156 34 35 37 73 77 78 82 79 80 81 3 4 5 133
169 0

Ścieżka dla grafu rbg358.atsp o długości 1213:

358

0 190 322 260 223 37 326 168 156 60 296 331 229 309 96 154 22 115 164 218 312 216 286 208 220
250 215 338 251 140 110 99 49 98 63 148 167 25 38 178 320 310 4 44 262 332 300 234 199 209 142
352 290 240 204 33 48 354 53 228 71 16 203 74 253 136 85 268 174 56 23 185 57 194 67 245 127
131 344 291 239 69 305 91 132 104 5 171 273 122 303 301 150 101 10 27 277 302 281 324 210 265
59 125 214 51 78 333 129 75 212 217 201 166 163 145 246 81 247 237 355 317 266 43 12 233 19
357 325 211 207 66 14 159 121 13 3 242 323 351 197 161 152 64 232 41 278 24 263 282 200 158
149 241 311 116 155 107 289 169 348 292 157 26 343 313 92 275 191 276 192 238 182 306 336 105
119 244 151 143 89 230 8 327 103 68 124 298 248 264 341 280 236 102 258 139 106 181 114 97 297
304 77 42 198 45 7 15 50 108 95 162 133 153 65 314 86 47 111 195 339 342 135 39 255 184 259 329
79 126 293 285 283 118 270 337 40 235 202 225 294 175 307 279 308 353 287 144 123 120 177 340
274 34 9 249 70 2 54 100 36 29 189 295 193 117 345 94 221 267 28 213 186 346 183 176 170 93 30
347 288 256 318 226 187 205 272 17 87 83 90 21 222 316 224 128 11 6 284 188 130 349 299 231
328 61 179 173 321 356 141 227 315 146 219 196 137 76 1 134 82 319 243 35 18 271 55 138 113 84
32 31 52 20 350 180 165 206 46 112 330 257 254 160 147 252 335 269 88 58 261 73 62 72 172 109
334 80 0